UNIVERSIDAD DE MANIZALES

Facultad de Ciencias e Ingenierías

Programación I

Actividad Colaborativa N.° 2
Evolución del Sistema de Gestión Académica

Documento de Decisiones de Diseño

Integrantes

Ómar Alejandro Gómez Rocha
Juan Carlos Girón Sanabria
Cristhian Mauricio Hernández Gaviria

Docente

Natalia Castellanos Gómez

Repositorio: github.com/OmarAlejandroGomez/Actividad_01
Etiquetas: Entrega_AC1 y Entrega_AC2

Noviembre de 2026

1. Introducción

Este documento explica y justifica las decisiones de diseño tomadas para evolucionar el Sistema de Gestión Académica construido en la Actividad Colaborativa N.° 1. La nueva versión incorpora tres conceptos de la programación orientada a objetos: clases abstractas, composición y manejo de excepciones personalizadas.

La versión de la Actividad 1 quedó identificada con la etiqueta `Entrega_AC1`, y la versión actual con la etiqueta `Entrega_AC2`.

El documento describe primero la solución en conjunto y luego desarrolla, con referencia directa a las clases y métodos del código, cuatro decisiones: la abstracción de la clase `Persona`, la organización de la oferta académica mediante grupos y horarios, el control de situaciones excepcionales durante la matrícula y la continuidad del polimorfismo.

2. Descripción general de la solución implementada

El sistema es una aplicación de consola en Python que registra programas, asignaturas y personas, y ahora también organiza grupos académicos con su horario y la matrícula de estudiantes. Se conservó la estructura de la primera entrega: los modelos están en la carpeta `models`, las colecciones de objetos en `controllers` y la interacción con el usuario en `main.py`.

Archivo	Responsabilidad	Cambio en la AC2
`models/persona.py`	Clase base de toda persona	Se convierte en clase abstracta
`models/estudiante.py`, `docente.py`, `administrativo.py`	Clases especializadas	Implementan el método abstracto
`models/horario.py`	Día, hora de inicio, hora de fin y salón	Nueva
`models/grupo_academico.py`	Grupo de una asignatura con docente, cupo, estudiantes y horario	Nueva
`models/excepciones.py`	Cinco excepciones personalizadas	Nueva
`controllers/grupo_controller.py`	Registro de grupos	Nueva
`controllers/asignatura_controller.py` y `programa_controller.py`	Registro y búsqueda	Se agregan `buscar_asignatura()` y `buscar_programa()`
`controllers/persona_controller.py`	Colección única de personas	Se agrega `get_docentes()`
`main.py`	Menú y demostración	Opciones 12 a 17 para grupos y búsquedas; la opción 18 ejecuta las pruebas mínimas

El menú permite crear grupos, matricular y retirar estudiantes, consultar los grupos y buscar asignaturas y programas por código.

3. Generalización de las personas

3.1 Por qué Persona es una clase abstracta

En la universidad nadie se vincula solo como “persona”, siempre es estudiante, docente o administrativo. Por eso un objeto `Persona` genérico no representa nada real dentro del sistema y permitir crearlo abriría la puerta a registros incompletos.

Para impedirlo, `Persona` hereda de `ABC`, del módulo `abc` de Python (Abstract Base Class), y su método `realizar_actividad_principal()` se declaró con `@abstractmethod`. Una clase con métodos abstractos no se puede instanciar: la instrucción `Persona("9999", "Sin Tipo", "sintipo@correo.com")` lanza un `TypeError`. La demostración de `main.py` intenta esa creación, captura el error y muestra el mensaje de rechazo.

`Persona` conserva lo que sí es común a todos: los atributos `identificacion`, `nombre` y `correo`, sus métodos `get_` y `set_` con validaciones, y `mostrar_informacion()`. Así la clase abstracta no es una cáscara vacía: concentra el comportamiento compartido y deja abierto solo lo que depende del tipo de persona. El constructor también quedó marcado como abstracto; como cada clase hija define su propio `__init__` y llama a `super().__init__(identificacion, nombre, correo)`, esto refuerza que la inicialización se completa siempre desde una clase especializada.

3.2 Cómo implementan el método abstracto las clases derivadas

Como el método es abstracto, cada clase hija está obligada a implementarlo; si no lo hiciera, tampoco podría instanciarse. Cada implementación usa los atributos propios de su clase para describir su responsabilidad:

Clase	Atributos que usa	Comportamiento de realizar_actividad_principal()
`Estudiante`	Programa académico, semestre, promedio acumulado	El estudiante realiza su actividad de estudio en su programa, el semestre que cursa y su promedio
`Docente`	Facultad, tipo de contratación, horas semanales	El docente realiza su actividad de docencia en su facultad, con su tipo de contrato y horas semanales
`Administrativo`	Dependencia, cargo, jornada	El administrativo realiza su actividad en su dependencia, con su cargo y jornada

Las tres clases también sobrescriben `mostrar_informacion()`: llaman a `super().mostrar_informacion()` para reutilizar los datos comunes y les agregan los propios (código estudiantil o número de empleado, entre otros). Los códigos `EST-001`, `Docente-001` y `ADM-001` se generan con un contador de clase y no se piden al usuario.

4. Organización de la oferta académica

4.1 Relaciones entre GrupoAcademico, Asignatura, Docente, Estudiante y Horario

`GrupoAcademico` es la clase que reúne la oferta: representa una asignatura que se dicta en un grupo concreto. Para ello se relaciona con las demás clases de tres maneras distintas, y cada una responde a una decisión de diseño.

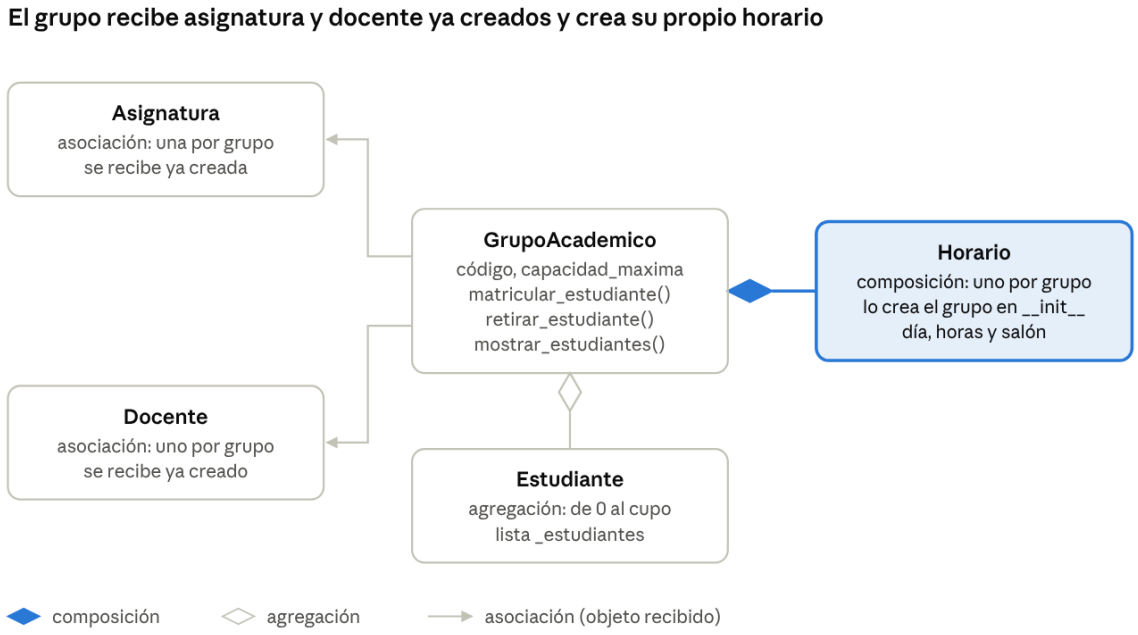


Figura 1. Relaciones de GrupoAcademico: asignatura, docente y estudiantes existen fuera del grupo; el horario solo existe dentro de él.

Asociación con Asignatura y Docente. El constructor `GrupoAcademico(codigo, asignatura, docente, capacidad_maxima, dia, hora_inicio, hora_fin, salon)` recibe objetos ya creados, no sus nombres ni sus códigos como texto. Los métodos `set_asignatura()` y `set_docente()` lo garantizan con `isinstance(asignatura, Asignatura)` e `isinstance(docente, Docente)`; si llega otro tipo de dato, lanzan un `ValueError`. Gracias a esto, `mostrar_informacion_grupo()` consulta directamente `get_nombre()` y `get_codigo()` de esos objetos. Estos objetos existen por sí mismos: una asignatura puede ofrecerse en varios grupos y un docente puede dictar en varios.

Agregación de Estudiante. El grupo guarda en la lista privada `_estudiantes` los objetos `Estudiante` matriculados. Esa lista solo cambia por medio de `matricular_estudiante(estudiante)` y `retirar_estudiante(codigo_estudiantil)`, que controlan el cupo y los duplicados; `get_estudiantes()` devuelve una copia para evitar modificaciones externas. Retirar a un estudiante del grupo no lo elimina del sistema: sigue registrado en `PersonaController`.

Composición con Horario. El grupo no recibe un horario ya hecho: lo crea dentro de su propio constructor con `self._horario = Horario(dia, hora_inicio, hora_fin, salon)`. `Horario` valida

el día y el salón, convierte las horas del formato `HH:MM` a objetos de tiempo y verifica que la hora de inicio sea anterior a la de finalización.

Para crear y guardar los grupos se agregó `GrupoController`, con `add_grupo()` y `get_grupos()`, siguiendo el mismo patrón de los controladores existentes. `PersonaController.get_docentes()` permite elegir el docente responsable desde el menú.

4.2 Por qué la relación entre grupo y horario es composición y no herencia

La herencia expresa una relación "es un": un `Docente` es una `Persona`. Entre grupo y horario esa frase no tiene sentido, porque un grupo académico no es un horario. Lo correcto es decir que un grupo tiene un horario, y esa relación "tiene un" se modela con un atributo que contiene otro objeto.

Si `GrupoAcademico` heredara de `Horario`, recibiría métodos como `set_rago_horas()` o `get_salon()` como si fueran propios, mezclaría dos responsabilidades en una sola clase e impediría que el grupo heredara de otra clase si más adelante lo necesitara.

La relación es composición, y no una asociación, por tres razones observables en el código:

1. **El horario nace dentro del grupo.** Se construye en `GrupoAcademico.__init__` a partir de los datos recibidos; no existe un horario creado por fuera que luego se asigne.
2. **Pertenece a un solo grupo.** Ningún otro objeto guarda una referencia a ese horario, y el grupo lo expone por medio de `get_horario()`.
3. **Comparten el ciclo de vida.** Si el grupo desaparece, su horario desaparece con él.

Además, representar el horario como objeto, y no como una cadena de texto, permite validar cada dato por separado y presentarlo de forma uniforme con `Horario.mostrar_informacion()`.

5. Control de situaciones excepcionales

5.1 Decisión: excepciones propias en lugar de mensajes con condicionales

En la primera entrega los errores de validación se manejaban con `ValueError`. Para la matrícula y las búsquedas se crearon cinco excepciones propias en `models/excepciones.py`. Cada una hereda de `Exception` y construye su mensaje en el constructor con `super().__init__(...)`, usando los datos del caso (código del estudiante, del grupo, de la asignatura o del programa). Así el mensaje se escribe una sola vez y queda listo para mostrarse con `print(error)`.

Por ejemplo, la excepción que controla la matrícula duplicada:

```
class EstudianteYaMatriculadoError(Exception):

    def __init__(self, codigo_estudiantil, codigo_grupo):

        super().__init__(f"Estudiante {codigo_estudiantil} ya está matriculado
en el grupo {codigo_grupo}")
```

Tener una clase por situación permite que `main.py` capture cada caso por su nombre (`except CapacidadMaximaError`) y que el código deje claro qué falló, algo que un `ValueError` genérico no distingue.

5.2 Qué controla cada excepción y dónde se genera

Cada excepción se lanza en el método que detecta la situación y se captura en `main.py`, tanto en las opciones del menú como en la demostración (opción 18).

Excepción	Situación que controla	Se lanza en	Se captura en main.py
`EstudianteYaMatriculadoError`	El estudiante ya está matriculado en el grupo	`GrupoAcademico.matricular_estudiante()`, al encontrar el mismo código estudiantil en `_estudiantes`	`matricular_en_grupo()`
`CapacidadMaximaError`	El grupo alcanzó su capacidad máxima	`GrupoAcademico.matricular_estudiante()`, cuando los matriculados igualan `capacidad_maxima`	`matricular_en_grupo()`
`EstudianteNoMatriculadoError`	El estudiante que se desea retirar no está matriculado	`GrupoAcademico.retirar_estudiante()`, después de recorrer toda la lista sin encontrarlo	`retirar_de_grupo()`
`AsignaturaNoEncontradaError`	La asignatura buscada no existe	`AsignaturaController.buscar_asignatura()`	`agregar_grupo()`, `buscar_asignatura()`
`ProgramaNoEncontradoError`	El programa académico buscado no existe	`ProgramaController.buscar_programa()`	`buscar_programa()`

5.3 Continuidad de la ejecución

Cada captura está dentro de un bloque `try` / `except` que imprime el mensaje con el prefijo “Error:” o “Error controlado:” y deja que el programa siga. En el menú, el usuario vuelve a la lista de opciones; en la demostración, la ejecución pasa al siguiente punto. Así, muestra los grupos con sus estudiantes después de todas las excepciones, lo que evidencia que el estado del sistema quedó intacto.

La búsqueda de asignaturas y programas se ubicó en los controladores porque son ellos los que guardan las listas y, por lo tanto, los que detectan que un código no existe. Los errores de captura de datos inválidos (campos vacíos, créditos o semestres fuera de rango) se siguen controlando con `ValueError`, como en la primera entrega.

6. Continuidad del polimorfismo

El polimorfismo de la primera entrega se conserva sin cambios en su funcionamiento y queda reforzado por la clase abstracta.

La colección única se mantiene. `PersonaController` sigue guardando estudiantes, docentes y administrativos en una sola lista, `personas`, por medio de `_agregar_persona()`, que valida la identificación sin importar el tipo de persona.

El recorrido polimórfico se mantiene. La función `mostrar_personas()` de `main.py` recorre esa lista con un solo ciclo y llama a `persona.mostrar_informacion()` y `persona.realizar_actividad_principal()` sin preguntar de qué clase es cada objeto. Python ejecuta en cada caso la versión de la clase real del objeto, de modo que un estudiante describe su estudio, un docente su docencia y un administrativo su labor en la dependencia.

La clase abstracta convierte ese polimorfismo en un contrato. En la primera versión, una clase hija podía olvidar implementar `realizar_actividad_principal()` y el error solo aparecía al ejecutar el ciclo. Ahora el método es abstracto, así que toda clase que herede de `Persona` debe implementarlo para poder crear objetos. El ciclo de `mostrar_personas()` tiene garantizado que cualquier elemento de la lista sabe responder a ese mensaje.

Los grupos aprovechan la misma jerarquía. `GrupoAcademico` trabaja con objetos `Docente` y `Estudiante`, que son también `Persona`, y usa sobre ellos métodos heredados como `get_nombre()`. Por su parte, `PersonaController.get_docentes()` obtiene los docentes de la misma colección única filtrándolos con `isinstance(persona, Docente)`, sin crear listas separadas por tipo.

7. Conclusiones

- **La clase abstracta hace cumplir una regla del dominio.** Convertir `Persona` en abstracta y declarar `realizar_actividad_principal()` como método abstracto impide registrar personas sin tipo y obliga a cada especialización a definir su comportamiento, sin perder los atributos y métodos comunes.
- **Cada relación se modeló según su naturaleza.** El grupo se asocia con objetos `Asignatura` y `Docente` que existen por sí mismos, agrega los `Estudiante` matriculados y se compone con su `Horario`, que nace y muere con él. La herencia se reservó para las relaciones “es un”.
- **Las excepciones personalizadas separan la detección del error de su presentación.** El modelo y los controladores lanzan la excepción donde detectan el problema; `main.py` la captura, muestra un mensaje comprensible y continúa. Las once pruebas mínimas se pueden verificar con la opción 18 del menú.
- **La evolución se hizo sobre el proyecto existente.** Se conservaron la estructura de modelos, controladores y menú, el estilo de getters y setters con validación y el recorrido polimórfico de la primera entrega, que queda identificada en el repositorio con la etiqueta `Entrega_AC1`.